

Study Notes: Heap Data Structure, Priority Queue & Heapsort

1. Motivation: Why do we need Heaps?

Imagine you are given a stream of numbers and your task is to **always be able to find the smallest (or largest) number** as fast as possible.

- **Approach 1: Unsorted Array:** Inserting is $O(1)$, but finding the minimum takes $O(N)$ (Linear Search).
- **Approach 2: Sorted Array:** Finding the minimum is $O(1)$ (it's at index 0), but inserting a new element and maintaining the sorted order takes $O(N)$ in the worst case.

The Heap Solution: Heaps hit the sweet spot. A heap allows you to get the conditioned item (e.g., the minimum or maximum element) in $O(1)$ **constant time** while keeping the cost of inserting a new element down to $O(\log N)$.

2. What is a Heap? (Structure & Properties)

Internally, a heap is **stored as an Array** (or ArrayList in Java), but it is **represented logically as a Tree**.

The Tree Representation:

A heap is visually structured as a **Complete Binary Tree**:

1. Every level of the tree is completely full, except possibly the last level.
2. The last level is filled strictly from **left to right**.
3. Because it's complete, the height of the tree is always $\log(N)$.

Heap Properties:

There are two main types of heaps based on the condition they maintain:

- **Max-Heap:** Every parent node is **greater than or equal to** all of its children. The maximum item is always at the root (index 0).
- **Min-Heap:** Every parent node is **smaller than or equal to** all of its children. The minimum item is always at the root (index 0).

Note: The array underneath does NOT need to be fully sorted. The only guarantee is the parent-child relationship.

3. The Array Formulas (No Pointers Needed!)

Because the heap is a complete binary tree stored in an array, we don't need `node.left` or `node.right` pointers. We find relationships using pure math based on the array index `i`.

If using a 0-indexed array (Standard in code):

- **Parent Node:** $(i - 1) / 2$
- **Left Child:** $(2 * i) + 1$
- **Right Child:** $(2 * i) + 2$

(Note: If using a 1-indexed array, Parent is $i/2$, Left is $2*i$, Right is $2*i + 1$).

4. Core Operations & Algorithms

A. Insertion (upheap) — Time Complexity: $O(\log N)$

When you add a new item, you must maintain the complete binary tree structure and the heap property.

1. **Add** the new element to the very end of the array (which conceptually places it at the bottom-left-most available spot in the tree).
2. **Compare** the newly added element with its parent.
3. **Swap** if it violates the heap property (e.g., in a Min-Heap, if the new child is smaller than the parent, swap them).
4. **Repeat** this upward comparison (upheap) until the condition is met or you reach the root. *Because the height of the tree is $\log(N)$, you make at most $\log(N)$ swaps.*

B. Deletion / Removal (downheap) — Time Complexity: $O(\log N)$

In a heap, you typically only remove the root element (the max or min).

1. **Remove** the root element (index 0). This is the value you return.
2. **Replace** the root with the very **last element** in the array.
3. **Remove** the last element from the array to shrink the size.
4. **Compare** the new root with its children.
5. **Swap** with the smaller child (for a Min-Heap) or larger child (for a Max-Heap).
6. **Repeat** this downward comparison (downheap) until the element is smaller/larger than its children or hits a leaf.

5. Heap Sort — Time Complexity: $O(N \log N)$

Heapsort is incredibly simple once you have a working Heap:

1. Take an unsorted array and insert all elements into a Heap.
2. Continuously remove the top element (using the deletion method).
3. Since the heap always returns the smallest (or largest) element, removing them one by one gives you a perfectly sorted sequence.
4. **Math:** Removing 1 item takes $O(\log N)$. Removing N items takes $O(N \log N)$.

6. What is a Priority Queue?

A Priority Queue is an Abstract Data Type, and **a Heap is the standard way to implement it.**

- "Priority" means the item with the highest priority (e.g., the smallest number) is served first.
- You *could* implement a Priority Queue with a Linked List by inserting items in sorted order, but insertion would take $O(N)$. Heaps are used because they handle insertions and retrievals much more efficiently ($O(\log N)$ and $O(1)$ respectively).

7. Advanced: Building a Heap from an Unsorted Array (Heapify)

If you are handed an entire unsorted array and told to make it a Max-Heap, you *could* insert elements one by one ($O(N \log N)$ total). But there is a faster way!

The $O(N)$ Heapify Approach:

1. In a complete binary tree, all the leaf nodes make up exactly half of the tree ($N / 2$ to N).
2. A single node with no children is technically already a valid heap. Therefore, **we can completely ignore the leaf nodes.**
3. Start from index $(N / 2)$ and iterate backwards down to index 0 .
4. Call the `downheap()` function on each of these nodes.
5. **Time Complexity:** Even though it looks like $(N/2) * O(\log N)$, the mathematical proof (using geometric series) bounds this to **strictly $O(N)$ time.**